MemProbe: Using Language to Probe Robot Memory

Research Proposal

Feiyang Wu

May 2026

1 Introduction & Why Memory is Important

Robot foundation models increasingly need memory because many manipulation tasks are not fully determined by the current observation. Objects may become occluded, containers may be opened and closed, goals may depend on previous subtasks, and repeated actions may require counting over time. In these settings, the robot must remember action-relevant facts such as where an object was placed, which drawer contained an item, whether a container was already used, or what progress has been completed. Without such memory, a policy may fail even when its current perception and low-level control are strong.

However, existing memory-augmented robot policies are usually evaluated through downstream task success, which only indirectly reflects memory quality. A higher success rate does not prove that the model remembered the right information, and a failure does not reveal whether the cause was memory, perception, planning, or control. MemProbe addresses this gap by using controlled question answering as a diagnostic interface for robotic memory. Instead of asking a memory model to reconstruct past frames at the pixel level, MemProbe asks whether it can answer action-relevant questions about past spatial states, temporal events, object locations, state changes, repeated actions, and subtask progress. In this way, MemProbe evaluates what a robot memory module actually stores, retains, and retrieves.

2 Research Motivation

Current memory-augmented Vision-Language-Action models are primarily evaluated through downstream task success rates, such as overall success rate, stage-wise success rate, or long-horizon completion rate. **While success rate metrics are useful for measuring whether a robot can complete a task, they provide only indirect evidence about the quality of the memory**

mechanism itself. A model may achieve higher task success because of better perception, stronger action priors, improved language grounding, or dataset bias, rather than because its memory module genuinely stores and retrieves useful past information. This raises a fundamental question: **How can we determine whether a robotic memory model actually remembers the information that matters?**

One possible way to test memory is to attach the memory module to a generative model and evaluate whether it can reconstruct a previously observed scene at the pixel level. However, this formulation may be unnecessarily strict and potentially misaligned with the role of memory in robotic decision-making. Robots do not always need to remember every visual detail of a past observation. In many manipulation tasks, the useful memory is not pixel-level appearance, but higher-level information: where objects were located, which objects were interacted with, what changed after an action, which containers were opened, what subgoals were completed, and what relevant constraints were observed earlier in the episode.

This motivates a different view: **Language can serve as a high-level abstraction of spatial-temporal memory**. Instead of asking whether a memory module can reconstruct an exact image, we can ask whether it can answer action-relevant questions about past observations and events. For example, a robot may need to remember which drawer contained the spoon, whether the red cup was moved, which object was placed into the bowl, or what the workspace looked like before a manipulation step. These questions directly probe whether the model preserves information that is useful for embodied reasoning and future action.

Therefore, the goal of this research is not to train a stronger Vision-Language-Action policy, nor to use language as an additional modality for improving generalization in the traditional VLA sense. Rather, the goal is to establish **an evaluation pipeline for measuring the memory quality of memory based robot foundation models**. In this pipeline, language is used as a diagnostic interface: it converts spatial-temporal memory into explicit, interpretable question-answering behavior. By asking targeted questions about past layouts, object states, action history, and temporal changes, we can evaluate whether a memory model stores, retains, and retrieves the information that is actually relevant to robot manipulation.

This perspective separates memory evaluation from policy execution. Instead of judging memory only through final task success, we directly test the content of memory itself. Such an evaluation can reveal whether a memory module remembers useful action-relevant facts, whether it forgets important historical information over time, whether it confuses similar objects or locations, and whether it can reason over temporal changes in the scene. In this way, language-based episodic memory question answering provides a more interpretable and targeted framework for evaluating robotic memory than task success rate alone.

3 Related Works

3.1 Robot Policies with Memory

Recently researches have been focusing on robot memory than ever before. MemoryVLA [1] proposes a cognition-memory-action framework for long-horizon manipulation. It maintains working memory from perceptual and cognitive tokens, stores consolidated low-level and high-level information in a perceptual-cognitive memory bank, retrieves decision-relevant memory entries, and conditions a diffusion action expert on the retrieved memory. Other recent policies make similar arguments from different architectural perspectives. VPWEM [2] introduces a visuomotor policy with both working memory and episodic memory. It keeps a sliding window of recent observation tokens as short-term memory and compresses older observations into a fixed number of episodic memory tokens through a Transformer-based contextual memory compressor. RoboMME [3] studies memory-augmented VLA policies more systematically by evaluating multiple $\pi_{0.5}$-based memory variants under temporal, spatial, object, and procedural memory demands. RoboMemory [4] uses memory-augmented embodied agent framework that integrates spatial, temporal, episodic, and semantic memory modules to improve long-horizon robotic task execution. Physical Intelligence [5] proposes multi-scale embodied memory, combining short-horizon video memory with long-horizon text memory for extended real-world robot tasks. VQ-Memory [6] represents past proprioceptive and temporal context with compact vector-quantized memory tokens. Keyframe-Chaining VLA [7] retrieves task-relevant historical keyframes and injects them into the VLA as visual memory tokens. Chameleon uses geometry-grounded multimodal episodic memory to handle perceptual aliasing in long-horizon manipulation.

Together, these works show that memory is becoming an important architectural component of robotic foundation models. However, their evaluation protocols remain largely policy-centric. Memory modules are typically judged by overall task success rate, stage-wise success rate, robustness under occlusion, or generalization to longer horizons. These metrics are necessary but indirect: they do not reveal whether the memory module actually stores and retrieves the action-relevant facts needed for decision-making. A policy may succeed because of stronger perception, better action priors, or shortcut correlations, while a policy may fail due to control errors even if its memory is correct. MemProbe is designed to complement this line of work by directly probing the content of robotic memory through language-based questions about past object locations, state changes, source-target mappings, repeated actions, and subtask progress.

3.2 Robot Control and Memory Benchmarks

RoboMemArena [8] explicitly studies memory requirements in robotic manipulation by constructing 26 manipulation tasks around four types of memory demands: sequence memory, counting, occlusion, and transferring. Its tasks involve long trajectories, with an average length exceeding 1,000 steps, and a

large fraction of subtasks are labeled as memory-dependent. RoboMemArena evaluates memory-augmented architectures primarily through task success rate and condition-specific success rate. RoboMME [3] is another highly relevant benchmark for robotic memory evaluation. It targets long-horizon and history-dependent robotic manipulation, where agents must remember information such as repeated action counts, temporarily occluded objects, and task procedures. RoboMME introduces a standardized benchmark with 16 manipulation tasks organized around a taxonomy of temporal, spatial, object, and procedural memory. Beyond the benchmark itself, it also evaluates 14 memory-augmented VLA variants built on the $\pi_{0.5}$ backbone, allowing systematic comparison of different memory representations and integration strategies. Its key finding is that memory design is strongly task-dependent: different memory representations provide different advantages across tasks rather than yielding a universally dominant solution.

Other robot learning benchmarks provide important resources but do not directly isolate memory as a question-answering problem. LIBERO [9] is a lifelong robot learning benchmark designed to study declarative and procedural knowledge transfer across 4 suites and 130 tasks. RoboCasa365 [10] expands the scale of simulation-based robot learning to 365 tasks, 2,500 kitchen scenes, and over 2,200 hours of robot demonstrations. DROID [11], BridgeData V2 [12], and Open X-Embodiment [13] provide large-scale real-world robot trajectories and unified data interfaces. These datasets are valuable raw materials for building robot memory QA style evaluations, since they contain rich long-horizon robot experiences. Nevertheless, they do not explicitly decompose the question of whether past events are correctly remembered into independent memory-centric QA tasks.

3.3 Robotic VQA and High-Level Reasoning

The second line of work studies robotic visual question answering and high-level reasoning over robot trajectories. Robo2VLM [14] demonstrates that robot trajectories can be automatically converted into VQA data. It derives ground-truth answers from non-visual robot signals such as end-effector poses, gripper width, and force sensing, segments trajectories into manipulation phases, and generates multiple-choice questions involving spatial reasoning, goal-conditioned reasoning, and interaction understanding. Robo2VLM-1 contains 684,710 questions generated from 176k real-world robot trajectories.

RoboVQA [15] focuses more on high-level planning and long-horizon reasoning. It releases 829,502 robotic visual question-answering samples and incorporates real-robot intervention rate as part of its evaluation. These works are important for MemProbe because they demonstrate that converting robot data into QA supervision is a feasible and scalable research direction.

However, MemProbe differs in its core target. Existing robotic VQA datasets mainly ask whether a model can reason about robot scenes, actions, or plans. They do not fully isolate whether the queried information is something the robot must remember from earlier in the episode before taking future actions.

In particular, they do not consistently enforce that questions depend on long episode history rather than the current frame, a short observation window, or static scene understanding. MemProbe instead focuses on action-relevant past facts: what object was previously located where, what changed after an interaction, which container was opened, which object was moved, and what information is no longer visible but remains necessary for future reasoning.

3.4 Episodic Memory QA, Long-Video QA, and Embodied QA

Long-video understanding and memory-augmented video QA provide important methodological foundations for MemProbe. A central challenge in this line of work is that modern multimodal models cannot process arbitrarily long videos at full resolution. As a result, many methods introduce memory, compression, retrieval, or sparse frame selection mechanisms to preserve useful information from long visual histories.

MovieChat [16] is an early representative example of this direction. It proposes a sparse memory mechanism for long-video understanding, inspired by human memory models, and uses memory tokens to reduce the cost of processing long visual sequences. MovieChat+ [17] further introduces question-aware sparse memory for long-video question answering, emphasizing that the memory used for answering a question should depend on the query itself. MA-LMM [18] similarly processes videos in an online manner and stores historical video information in a memory bank, allowing a multimodal LLM to refer back to previous video content without exceeding context-length or GPU-memory constraints. These works show that explicit memory mechanisms can help multimodal models handle long visual histories more efficiently.

Benchmark work in long-video QA further shows the importance of evaluating whether models can retrieve and reason over temporally distant evidence. EgoSchema [19] evaluates very long-form video-language understanding using more than 5,000 human-curated multiple-choice questions derived from Ego4D [20], and argues that long-video difficulty should be measured not only by clip length but also by the temporal evidence required to answer the question. LongVideoBench [21] introduces long-context interleaved video-language understanding, with videos up to one hour and 6,678 human-annotated multiple-choice questions, requiring models to retrieve and reason over relevant multimodal details from long inputs. CG-Bench [22] further emphasizes clue-grounded QA, arguing that answer accuracy alone may be insufficient because models can sometimes exploit multiple-choice shortcuts without truly grounding their answers in the correct video evidence.

These works are closely related to MemProbe because they treat memory as a mechanism for compressing, retrieving, and reasoning over long visual histories. However, their target domain is general video understanding rather than robot manipulation. Their questions usually focus on human activities, movie events, daily scenes, or general temporal reasoning. In contrast, MemProbe focuses on robot-action-relevant memory: object placements, source-target mappings,

container states, repeated manipulation actions, and subtask progress. The goal is not simply to determine whether a model understands a long video, but whether a robot memory module retains the specific past information needed for future action.

MemProbe therefore adopts the evaluation philosophy of long-video memory QA—especially long-horizon evidence retrieval, query-dependent memory access, and evidence-grounded evaluation—but shifts the task definition to robotic episodic memory. In this sense, long-video memory models provide useful technical inspiration, while MemProbe provides a robot-specific diagnostic benchmark for testing what a memory module actually remembers from an embodied manipulation history.

3.5 Positioning of MemRrobe

Taken together, the closest prior benchmark combination is not a single existing work, but a composition of several research threads. RoboMemArena provides a task-level formulation of robotic memory requirements. Robo2VLM provides a scalable precedent for translating robot trajectories into question-answering data. EMQA, Ego4D Episodic Memory, and OpenEQA provide the episodic-memory QA formulation. EgoSchema, LongVideoBench, MVBench, and the Perception Test provide mature practices for evaluating long-range video understanding, including evidence grounding and hidden evaluation.

MemProbe is positioned at the intersection of these lines of work. Its goal is not merely to build another robot task benchmark, another robotic VQA dataset, or another long-video QA benchmark. Instead, REM-QA aims to directly evaluate whether a robot foundation model remembers the action-relevant information it needs from past experience. By turning robot episode history into targeted memory questions, REM-QA separates memory evaluation from downstream policy success and provides a more interpretable way to test whether a memory model stores, retains, and retrieves useful spatial-temporal information.

4 Proposed Method

MemProbe should be built as a hybrid, provenance-preserving QA generation pipeline whose backbone is deterministic answer derivation from robot signals and simulator state, with LLMs used mainly for paraphrasing, formatting, and distractor synthesis rather than for inventing facts. The most productive primary sources are RoboCasa365 and LIBERO for simulation-scale grounded facts, RoboMemArena and RoboMME for memory-targeted task structure, and DROID plus BridgeData V2 for real-world trajectories. Robo2VLM is especially important as a methodological precedent because it already shows that end-effector pose, gripper aperture, and force sensing can be converted into phase-aware VQA generation on real robot trajectories. Open X-Embodiment is strategically valuable for scale, but it should be added only after a robust source-adapter layer exists because its constituent datasets are heterogeneous.

4.1 Design Principle

The central design principle is simple: MemProbe should measure remembered facts, not just downstream task completion. That implies three technical consequences. First, every QA pair must be tied to explicit supporting evidence in the episode history, similar to EMQA, Ego4D Episodic Memory, and grounded-video QA settings. Second, evaluation should include memory-specific diagnostics such as accuracy vs. evidence-query lag, occlusion-conditioned accuracy, and performance under explicit memory budgets, inspired by long-video memory work such as MovieChat+ and MA-LMM. Third, quality control has to be more ambitious than ordinary dataset cleaning: temporal leakage tests, evidence necessity/sufficiency tests, cross-modal consistency checks, and stratified human audits are all necessary if MemProbe is to be a trustworthy diagnostic benchmark.

The ordering that follows from these sources is straightforward. Phase one should be simulation-first: RoboCasa365, LIBERO, RoboMemArena, and optionally RoboMME. Phase two should add real-world signal-grounded generation from DROID and BridgeData V2, using Robo2VLM-like phase segmentation. Phase three should scale through Open X-Embodiment once a canonical adapter and provenance layer exist. RoboVQA and the non-robot long-video benchmarks should primarily shape interface design, evaluation design, and baseline selection rather than dominate the generated corpus.

4.2 Pipeline logic

MemProbe aims to evaluate whether a robot memory module retains action-relevant spatial-temporal information from previous observations. To avoid conflating memory failure with ambiguous language generation, MemProbe uses a structured-to-language QA generation pipeline. Each question is generated from an underlying symbolic memory query, and the natural-language question is only a controlled surface realization of that query. The main evaluation format is multiple-choice QA with canonical structured answers, evidence frames, explicit temporal anchors, and spatial reference-frame metadata.

The pipeline consists of eight stages:

- Robot trajectory
- Canonical episode representation
- Temporal event graph + spatial scene graph
- Memory fact graph
- Structured memory query generation
- Controlled multiple-choice QA generation
- Ambiguity and leakage filtering
- Human verification and benchmark release

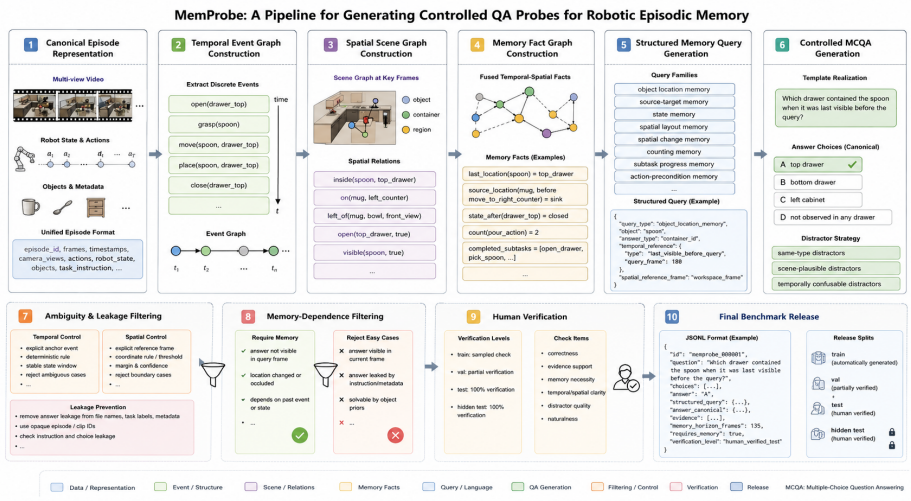

Figure 1: Pipeline Logic

4.2.1 Canonical Episode Representation

The first step is to normalize heterogeneous robot datasets into a shared episode format. Each source dataset may provide different information: simulator states, RGB videos, robot actions, gripper state, proprioception, language instructions, object poses, or task metadata. MemProbe converts these inputs into a canonical representation:

```
{
  "episode_id": "ep_000001",
  "source_dataset": "robocasa",
  "frames": [],
  "timestamps": [],
  "camera_views": ["front", "wrist"],
  "robot_state": [],
  "actions": [],
  "objects": [],
  "task_instruction": "",
  "source_metadata": {}
}
```

For simulation datasets such as RoboCasa and LIBERO, object poses, container states, and task predicates can be extracted from simulator state. For real-world datasets such as DROID or BridgeData, the representation may be constructed from RGB videos, gripper state, end-effector motion, language labels, object detectors, trackers, and optional depth or multi-view information.

The goal of this stage is not yet to generate questions, but to create a dataset-independent interface for memory fact extraction.

4.2.2 Temporal Event Graph Construction

The next step is to extract a temporal event graph from each episode. This graph records discrete manipulation events and their temporal boundaries.

Typical event types include:

```
grasp(object)
release(object)
place(object, receptacle_or_surface)
move(object, source, target)
open(container)
close(container)
pour(source, target)
push(object)
toggle(object)
complete_subtask(subtask)
```

Each event is represented as:

```
{
  "event_id": "place_butter_cabinet_001",
  "event_type": "place",
  "object": "butter",
  "target": "cabinet_2",
  "start_frame": 84,
  "end_frame": 96,
  "pre_state": {},
  "post_state": {}
}
```

For simulation data, these events can be extracted from state transitions, task predicates, contact states, object poses, and container open/close values. For real-world data, events can be inferred from gripper open/close transitions, object motion relative to the gripper, end-effector trajectories, contact cues, and object tracking.

This temporal graph supports sequential memory questions, such as:

```
Which object did the robot pick up first?
How many times did the robot pour?
Which subtask had already been completed before the query?
Where was the mug immediately before it was moved to the right counter?
```

4.2.3 Spatial Scene Graph Construction

Temporal events alone are not sufficient for MemProbe because many important memory questions are spatial rather than purely sequential. Therefore, MemProbe also constructs spatial scene graphs at selected keyframes or stable states.

A spatial scene graph contains object nodes, region nodes, container nodes, and spatial relations:

```
inside(spoon, top_drawer)
on(mug, left_counter)
in_workspace_zone(plate, sink_area)
left_of(red_cup, bowl, reference_frame=front_camera)
near(milk, fridge)
visible(spoon, true)
occluded(spoon, false)
reachable(top_drawer, true)
open(top_drawer, true)
```

A scene graph snapshot is represented as:

```
{
  "frame": 42,
  "stable": true,
  "objects": ["spoon", "mug", "bowl"],
  "relations": [
    ["inside", "spoon", "top_drawer"],
    ["on", "mug", "left_counter"],
    ["left_of", "mug", "bowl", "front_camera"],
    ["visible", "spoon", true],
    ["open", "top_drawer", true]
  ]
}
```

For robust benchmark construction, MemProbe prioritizes spatial relations that are stable and robot-action-relevant:

```
object-in-container
object-on-surface
workspace-zone membership
container open/closed state
last-visible location
source-target location
object visibility
```

Fine-grained relations such as *left_of*, *right_of*, *near*, or *closest_to* can be included, but only when their reference frame and coordinate rule are explicit.

4.2.4 Memory Fact Graph Construction

The temporal event graph and spatial scene graphs are then combined into a memory fact graph. This graph stores facts that a robot may need to remember for future action.

Examples include:

```
last_location(butter) = cabinet_2
source_location(mug, before move_to_right_counter) = sink
state_after_interaction(drawer_top) = closed
last_visible_location(spoon) = top_drawer
count(pour_action) = 2
completed_subtasks = [open_drawer, pick_spoon]
```

Unlike raw scene graphs, memory facts are query-oriented. They encode what the benchmark may ask about. Each fact includes:

```
{
  "fact_id": "fact_000213",
  "fact_type": "object_location_memory",
  "predicate": "inside",
  "object": "spoon",
  "value": "top_drawer",
  "temporal_reference": {
    "type": "last_visible_before_query",
    "query_frame": 180
  },
  "spatial_reference_frame": "workspace_frame",
  "evidence_frames": [42, 43, 44],
  "valid_query_frames": [120, 220]
}
```

This design separates the factual memory content from the eventual natural-language question.

4.2.5 Structured Memory Query Generation

Each QA item is first created as a structured query over the memory fact graph, not as free-form text.

Example:

```
{
  "query_type": "object_location_memory",
  "target_fact": "last_visible_location(spoon)",
  "object": "spoon",
  "answer_type": "container_id",
  "temporal_reference": {
    "type": "last_visible_before_query",
    "query_frame": 180
  },
  "spatial_reference_frame": "workspace_frame"
}
```

This structured query can later be rendered into a controlled natural-language question:

```
Which drawer contained the spoon when it was last visible before the query?
```

The important point is that the natural-language wording does not define the ground truth. The ground truth is defined by the structured query.

MemProbe supports several query families:

```
1. Object location memory
   - Where was object X when it was last visible?
   - Which container contained object X before the query?

2. Source-target memory
   - Where did object X come from before it was moved to target Y?
   - Which target did object X get placed into?

3. Container and object state memory
   - Was drawer X open or closed after the previous interaction?
   - Was object X already grasped or released before the query?

4. Spatial layout memory
   - Which workspace region contained object X at the beginning?
   - Which object was inside container Y when it was opened?

5. Spatial change memory
   - What changed about object X's location after the robot moved it?
   - Which object was no longer on the counter after the manipulation?

6. Counting and repetition memory
   - How many times did the robot pour?
   - How many objects had already been transferred before the query?

7. Subtask progress memory
   - Which subtask had already been completed before the query?
   - What remains to be done?

8. Action-precondition memory
   - Which previous event determines the next placement?
   - Which container should the robot avoid because it was already filled?
```

4.2.6 Controlled Multiple-Choice QA Generation

MemProbe uses multiple-choice QA as the primary evaluation format. This reduces answer-expression ambiguity and avoids relying on LLM-based free-form answer judging.

Each structured query is rendered into a controlled natural-language template. For example:

```
Template:
```

```
Which drawer contained <object> when it was last visible before the query?

Rendered question:
Which drawer contained the spoon when it was last visible before the query?
```

The choices are generated from a canonical answer ontology:

```
{
  "choices": [
    {"id": "A", "text": "top drawer", "canonical": "drawer_top"},
    {"id": "B", "text": "bottom drawer", "canonical": "drawer_bottom"},
    {"id": "C", "text": "left cabinet", "canonical": "cabinet_left"},
    {"id": "D", "text": "not observed in any drawer", "canonical": "none"}
  ],
  "answer": "A"
}
```

Distractors are not random. They are sampled from semantically compatible alternatives:

```
same-type distractors:
other drawers, cabinets, counters, containers

scene-plausible distractors:
    locations or objects that appear in the same episode

temporally confusable distractors:
    source location, target location, previous location, later location

hard negatives:
    containers or states that were interacted with but are not correct
```

For example:

```
Question:
Where was the mug immediately before the robot moved it to the right counter?

Correct:
sink

Distractors:
left counter      // plausible location
right counter     // temporally confusable target
cabinet shelf     // scene-plausible location
```

This makes the task harder than trivial answer-type matching.

4.2.7 Temporal Ambiguity Control

Temporal language is a major source of ambiguity. Words such as "earlier," "previously," "before," and "after" are not used as unconstrained ground-truth definitions. Each question must have a deterministic temporal reference rule.

Supported temporal reference types include:

```
episode_start
episode_end
before_event
after_event
during_event
last_visible_before_query
last_stable_state_before_query
first_occurrence
last_occurrence
subtask_boundary
```

For example:

```
{
  "temporal_reference": {
    "type": "before_event",
    "anchor_event_id": "move_mug_to_right_counter_001",
    "relation": "immediately_before",
    "state_rule": "last_stable_location_before_event_start",
    "stability_window_frames": 10
  }
}
```

Instead of asking:

```
Where was the mug earlier?
```

MemProbe asks:

```
Where was the mug immediately before the robot moved it to the right counter?
```

Items are rejected if:

```
the anchor event is ambiguous
the same event type occurs multiple times without instance specification
the relevant object changes state multiple times in the query window
the state is unstable near the anchor
different temporal interpretations produce different answers
the evidence span cannot be localized
```

4.2.8 Spatial Ambiguity Control

Spatial language is also ambiguous. Relations such as "left," "right," "near," and "next to" depend on reference frame and threshold. MemProbe therefore prioritizes robust spatial categories:

```
inside(container)
on(surface)
workspace zone
last visible location
source container
target container
open/closed state
```

When fine-grained spatial relations are used, they must specify:

```
reference frame
camera view or workspace coordinate system
coordinate rule
margin threshold
confidence score
evidence frame
```

Example:

```
{
  "relation": "left_of",
  "reference_frame": "camera_view",
  "camera_id": "front",
  "coordinate_rule": "bbox_center_x(mug) < bbox_center_x(bowl) - 40px",
  "ambiguity_margin_px": 40
}
```

A corresponding question would be:

```
In the front camera view, was the mug visually to the left or right of the bowl when the
```

rather than:

```
Was the mug on the left?
```

Items near relation boundaries are discarded. If a relation is inconsistent across views, MemProbe either rejects the item or makes the question explicitly view-specific.

4.2.9 Memory-Dependence Filtering

MemProbe should avoid questions that can be answered from the current frame alone. For each generated item, the pipeline checks whether the answer is visible or directly inferable at the query frame.

A QA item is considered memory-dependent if:

```
the answer object is no longer visible
the relevant container interior is occluded
the spatial relation has changed
the question asks about a prior stable state
the answer depends on a past event count
the answer depends on previous subtask completion
```

The pipeline rejects or labels as easy any item where:

```
the current frame alone reveals the answer
the answer is contained in the task instruction
the answer is leaked by the file name or metadata
the question can be answered from object priors
```

This step is essential for ensuring that MemProbe probes episodic memory rather than static perception.

4.2.10 Leakage Prevention

Because many robot datasets contain informative file names, task labels, or path names, MemProbe removes answer leakage from public annotations.

For example, a raw path such as: *place_butter_cabinet2_seed100.hdf5*

leaks the answer ‘cabinet2‘. Therefore, public benchmark files should use opaque identifiers:

```
{
  "episode_uid": "ep_83af91",
  "clip_uid": "clip_019b"
}
```

The mapping from opaque IDs to original dataset files can be stored separately in a local loader script, not exposed as part of model input.

Leakage filtering should check:

```
file paths
task instructions
episode names
object names
subtask labels
choice ordering
template artifacts
metadata fields given to the model
```

This is especially important if models are evaluated with text prompts that include dataset metadata.

4.2.11 Human Verification

MemProbe can use automatic generation for scale, but the test set should be human verified.

Recommended strategy:

```
Training split:
    automatically generated, with sampled quality checks

Validation split:
    automatically generated + partial human verification

Test split:
    100% human verified

Hidden test split:
    100% human verified, answers not publicly released
```

Human annotators should check:

```
whether the answer is correct
whether the evidence frame supports the answer
whether the question requires memory
whether temporal and spatial references are unambiguous
whether distractors are plausible but incorrect
whether the question is natural and understandable
```

Items with annotator disagreement should be removed or adjudicated.

4.2.12 Final JSONL Format

A final MemProbe item may look like:

```
{
  "id": "memprobe_000001",
  "source_dataset": "robocasa",
  "episode_uid": "ep_83af91",
  "query_frame": 180,

  "question_family": "object_location_memory",
  "question": "Which drawer contained the spoon when it was last visible before the query?",
  "choices": [
    {"id": "A", "text": "top drawer", "canonical": "drawer_top"},
    {"id": "B", "text": "bottom drawer", "canonical": "drawer_bottom"},
    {"id": "C", "text": "left cabinet", "canonical": "cabinet_left"},
```

```
    {"id": "D", "text": "not observed in any drawer", "canonical": "none"}
  ],
  "answer": "A",

  "structured_query": {
    "predicate": "location",
    "object": "spoon",
    "answer_type": "container_id",
    "temporal_reference": {
      "type": "last_visible_before_query",
      "query_frame": 180,
      "state_rule": "location_at_latest_visible_frame"
    },
    "spatial_reference_frame": "workspace_frame"
  },

  "answer_canonical": {
    "predicate": "inside",
    "object": "spoon",
    "container": "drawer_top"
  },

  "evidence": [
    {
      "type": "spatial_snapshot",
      "frame_range": [42, 45],
      "fact": "inside(spoon, drawer_top)"
    }
  ],

  "memory_horizon_frames": 135,
  "requires_memory": true,
  "answerable_from_query_frame": false,
  "verification_level": "human_verified_test"
}
```


4.2.13 Summary

The key idea of the MemProbe generation pipeline is:

```
Do not ask an LLM to invent QA pairs from videos.
Instead:
extract temporal-spatial facts,
generate structured memory queries,
render them as controlled multiple-choice questions,
attach canonical answers and evidence,
filter ambiguity and leakage,
```

```
then verify the test set manually.
```

This design makes MemProbe more defensible as a benchmark. It reduces linguistic ambiguity, avoids free-form answer judging, supports both sequential and spatial memory, and directly evaluates whether a robot memory module stores and retrieves action-relevant past facts.

5 Ablation Study

We evaluate existing memory models by attaching them to a shared open-source VLM backbone, such as Qwen-VL [23], and testing them on MemProbe. The purpose is not to train a stronger VLA policy, but to evaluate whether different memory representations preserve action-relevant information that can be queried through language. **All memory model weights are frozen during evaluation.** When necessary, only a lightweight adapter is trained or used to map the memory representation into the VLM input space.

We compare the following settings:

1. No-memory baseline: the VLM receives only the current frame and the question.
2. Recent-frame baseline: the VLM receives the most recent (k) frames.
3. Random-frame baseline: the VLM receives (k) randomly sampled frames from the episode history.
4. Oracle-evidence baseline: the VLM receives the ground-truth evidence frame or span.
5. Latent-memory interface: the frozen memory model encodes the episode history into a compact latent representation, which is projected into the VLM embedding space.
6. Token-memory interface: the frozen memory model outputs memory tokens, retrieved keyframes, compressed visual tokens, or textual memory summaries, which are inserted into the VLM context.
7. Hybrid interface: both latent memory and explicit memory tokens are provided to the VLM.

The ablation of memory insertion into the model can reference Figure 2.

This ablation allows us to separate several factors: whether memory is useful at all, whether learned memory is better than simple frame sampling, whether explicit tokens are more effective than compact latent states, and whether a memory model exposes information in a form that a VLM can use. We evaluate each setting with the same VLM, prompt format, frame budget, and MemProbe split. The primary metric is multiple-choice accuracy, supplemented by

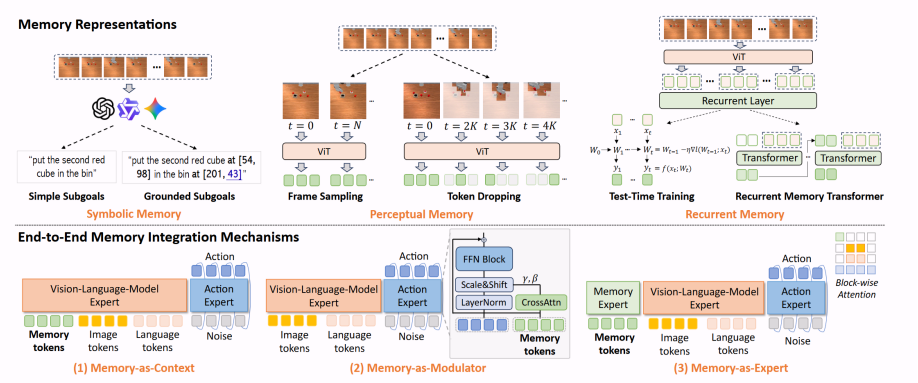

Figure 2: RoboMME [3] Memory Insertion Methods

category-wise accuracy, memory-horizon analysis, spatial versus temporal memory breakdown, and evidence-grounding performance.

A key hypothesis is that different memory interfaces will be useful for different types of questions. Latent memory may capture global task progress, repeated actions, or coarse temporal state, while token-based memory may better preserve object-level and spatial evidence. Oracle-evidence performance provides an upper bound on what the VLM can answer when the relevant memory is perfectly retrieved. The gap between oracle evidence and learned memory indicates how much information is lost during memory storage or retrieval.

References

[1] H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, T. Wang, E. Zhou, H. Fan, X. Zhang, and G. Huang, "Memoryvla: Perceptual-cognitive memory in vision-language-action models for robotic manipulation," 2026. [Online]. Available: https://arxiv.org/abs/2508.19236

[2] Y. Lei, Z. Liang, H. Zhang, and P. Luo, "Vpwem: Non-markovian visuomotor policy with working and episodic memory," 2026. [Online]. Available: https://arxiv.org/abs/2603.04910

[3] Y. Dai, H. Fu, J. Lee, Y. Liu, H. Zhang, J. Yang, C. Finn, N. Fazeli, and J. Chai, "Robomme: Benchmarking and understanding memory for robotic generalist policies," *arXiv preprint arXiv:2603.04639*, 2026.

[4] M. Lei, H. Cai, Y. Yang, Y. Wu, J. Ren, Z. Cui, L. Tan, J. Hong, G. Hu, S. Zhu, S. Jiang, G. Wang, J. Tan, Z. Wan, Z. Li, Z. Li, S. Cui, Y. Zhao, and Y. Han, "Robomemory: A brain-inspired multi-memory agentic framework for interactive environmental learning in physical embodied systems," 2026. [Online]. Available: https://arxiv.org/abs/2508.01415

[5] M. Torne, K. Pertsch, H. Walke, K. Vedder, S. Nair, B. Ichter, A. Z. Ren, H. Wang, J. Tang, K. Stachowicz, K. Dhabalia, M. Equi, Q. Vuong, J. T. Springenberg, S. Levine, C. Finn, and D. Driess, "Mem: Multi-scale embodied memory for vision language action models," 2026. [Online]. Available: https://arxiv.org/abs/2603.03596

[6] H. Wang, Z. Jing, J. Ao, S. Song, X. Li, G. Huang, and C. Bai, "Beyond short-horizon: Vq-memory for robust long-horizon manipulation in non-markovian simulation benchmarks," 2026. [Online]. Available: https://arxiv.org/abs/2603.09513

[7] Y. Chen, W. Tan, L. Zhu, F. Li, J. Li, G. Yang, and H. T. Shen, "Non-markovian long-horizon robot manipulation via keyframe chaining," 2026. [Online]. Available: https://arxiv.org/abs/2603.01465

[8] H. Lei, W. Song, H. Zhang, J. Pei, J. Chen, H. Yan, H. Zhao, P. Ding, Z. Zhang, L. Huang, D. Wang, Y. Wang, and H. Li, "Robomemarena: A comprehensive and challenging robotic memory benchmark," 2026. [Online]. Available: https://arxiv.org/abs/2605.10921

[9] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "Libero: Benchmarking knowledge transfer for lifelong robot learning," 2023. [Online]. Available: https://arxiv.org/abs/2306.03310

[10] S. Nasiriany, S. Nasiriany, A. Maddukuri, and Y. Zhu, "Robocasa365: A large-scale simulation framework for training and benchmarking generalist robots," 2026. [Online]. Available: https://arxiv.org/abs/2603.04356

[11] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, P. D. Fagan, J. Hejna, M. Itkina, M. Lepert, Y. J. Ma, P. T. Miller, J. Wu, S. Belkhale, S. Dass, H. Ha, A. Jain, A. Lee, Y. Lee, M. Memmel, S. Park, I. Radosavovic, K. Wang, A. Zhan, K. Black, C. Chi, K. B. Hatch, S. Lin, J. Lu, J. Mercat, A. Rehman, P. R. Sanketi, A. Sharma, C. Simpson, Q. Vuong, H. R. Walke, B. Wulfe, T. Xiao, J. H. Yang, A. Yavary, T. Z. Zhao, C. Agia, R. Baijal, M. G. Castro, D. Chen, Q. Chen, T. Chung, J. Drake, E. P. Foster, J. Gao, V. Guizilini, D. A. Herrera, M. Heo, K. Hsu, J. Hu, M. Z. Irshad, D. Jackson, C. Le, Y. Li, K. Lin, R. Lin, Z. Ma, A. Maddukuri, S. Mirchandani, D. Morton, T. Nguyen, A. O'Neill, R. Scalise, D. Seale, V. Son, S. Tian, E. Tran, A. E. Wang, Y. Wu, A. Xie, J. Yang, P. Yin, Y. Zhang, O. Bastani, G. Berseth, J. Bohg, K. Goldberg, A. Gupta, A. Gupta, D. Jayaraman, J. J. Lim, J. Malik, R. Martín-Martín, S. Ramamoorthy, D. Sadigh, S. Song, J. Wu, M. C. Yip, Y. Zhu, T. Kollar, S. Levine, and C. Finn, "Droid: A large-scale in-the-wild robot manipulation dataset," 2024.

[12] H. Walke, K. Black, A. Lee, M. J. Kim, M. Du, C. Zheng, T. Zhao, P. Hansen-Estruch, Q. Vuong, A. He, V. Myers, K. Fang, C. Finn, and

S. Levine, "Bridgedata v2: A dataset for robot learning at scale," 2024. [Online]. Available: https://arxiv.org/abs/2308.12952

[13] A. R. et al., "Open x-embodiment: Robotic learning datasets and rt-x models," 2025. [Online]. Available: https://arxiv.org/abs/2310.08864

[14] K. Chen, S. Xie, Z. Ma, P. R. Sanketi, and K. Goldberg, "Robo2vlm: Visual question answering from large-scale in-the-wild robot manipulation datasets," 2025. [Online]. Available: https://arxiv.org/abs/2505.15517

[15] P. Sermanet, T. Ding, J. Zhao, F. Xia, D. Dwibedi, K. Gopalakrishnan, C. Chan, G. Dulac-Arnold, S. Maddineni, N. J. Joshi, P. Florence, W. Han, R. Baruch, Y. Lu, S. Mirchandani, P. Xu, P. Sanketi, K. Hausman, I. Shafran, B. Ichter, and Y. Cao, "Robovqa: Multimodal long-horizon reasoning for robotics," 2023. [Online]. Available: https://arxiv.org/abs/2311.00899

[16] E. Song, W. Chai, G. Wang, Y. Zhang, H. Zhou, F. Wu, H. Chi, X. Guo, T. Ye, Y. Zhang, Y. Lu, J.-N. Hwang, and G. Wang, "Moviechat: From dense token to sparse memory for long video understanding," 2024. [Online]. Available: https://arxiv.org/abs/2307.16449

[17] E. Song, W. Chai, T. Ye, J.-N. Hwang, X. Li, and G. Wang, "Moviechat+: Question-aware sparse memory for long video question answering," 2024. [Online]. Available: https://arxiv.org/abs/2404.17176

[18] B. He, H. Li, Y. K. Jang, M. Jia, X. Cao, A. Shah, A. Shrivastava, and S.-N. Lim, "Ma-lmm: Memory-augmented large multimodal model for long-term video understanding," 2024. [Online]. Available: https://arxiv.org/abs/2404.05726

[19] K. Mangalam, R. Akshulakov, and J. Malik, "Egoschema: A diagnostic benchmark for very long-form video language understanding," 2023. [Online]. Available: https://arxiv.org/abs/2308.09126

[20] K. Grauman, A. Westbury, E. Byrne, Z. Chavis, A. Furnari, R. Girdhar, J. Hamburger, H. Jiang, M. Liu, X. Liu, M. Martin, T. Nagarajan, I. Radosavovic, S. K. Ramakrishnan, F. Ryan, J. Sharma, M. Wray, M. Xu, E. Z. Xu, C. Zhao, S. Bansal, D. Batra, V. Cartillier, S. Crane, T. Do, M. Doulaty, A. Erapalli, C. Feichtenhofer, A. Fragomeni, Q. Fu, A. Gebreselasie, C. Gonzalez, J. Hillis, X. Huang, Y. Huang, W. Jia, W. Khoo, J. Kolar, S. Kottur, A. Kumar, F. Landini, C. Li, Y. Li, Z. Li, K. Mangalam, R. Modhugu, J. Munro, T. Murrell, T. Nishiyasu, W. Price, P. R. Puentes, M. Ramazanova, L. Sari, K. Somasundaram, A. Southerland, Y. Sugano, R. Tao, M. Vo, Y. Wang, X. Wu, T. Yagi, Z. Zhao, Y. Zhu, P. Arbelaez, D. Crandall, D. Damen, G. M. Farinella, C. Fuegen, B. Ghanem, V. K. Ithapu, C. V. Jawahar, H. Joo, K. Kitani, H. Li, R. Newcombe, A. Oliva, H. S. Park, J. M. Rehg, Y. Sato, J. Shi, M. Z. Shou, A. Torralba, L. Torresani, M. Yan, and J. Malik, "Ego4d:

Around the world in 3,000 hours of egocentric video," 2022. [Online]. Available: https://arxiv.org/abs/2110.07058

[21] H. Wu, D. Li, B. Chen, and J. Li, "Longvideobench: A benchmark for long-context interleaved video-language understanding," 2024. [Online]. Available: https://arxiv.org/abs/2407.15754

[22] G. Chen, Y. Liu, Y. Huang, Y. He, B. Pei, J. Xu, Y. Wang, T. Lu, and L. Wang, "Cg-bench: Clue-grounded question answering benchmark for long video understanding," 2024. [Online]. Available: https://arxiv.org/abs/2412.12075

[23] J. Bai, S. Bai, S. Yang, S. Wang, S. Tan, P. Wang, J. Lin, C. Zhou, and J. Zhou, "Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond," 2023. [Online]. Available: https://arxiv.org/abs/2308.12966